



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

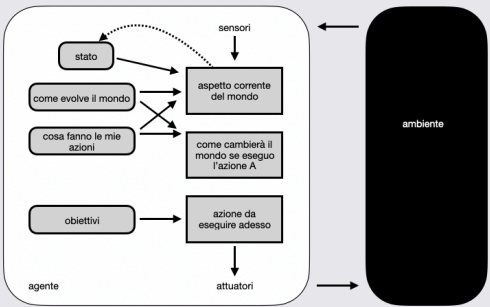
Lezione 4 - Algoritmi di ricerca non informata



Algoritmi di Ricerca Non Informata

Nella lezione precedente...

Agenti Basati su Obiettivi



Flessibilità

La conoscenza è rappresentata e può essere modificata, pertanto è da considerare come un agente più **flessibile** degli altri.

In un agente reattivo, dovremmo riscrivere molte delle regole condizione-azione; qui, invece, il comportamento può essere adattato.

Filosofia degli agenti basati su obiettivi

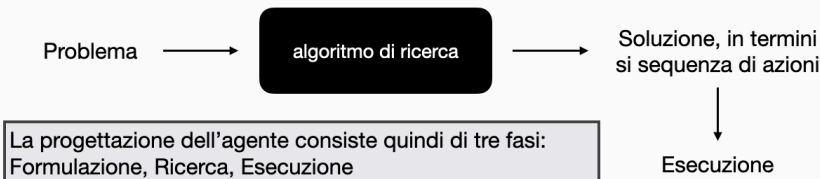
Talvolta, scegliere un'azione in base ad un obiettivo è molto semplice, quando questo può essere raggiunto in un solo passo. Altre volte è più difficile, ad esempio quando l'agente deve considerare lunghe sequenze di azioni per trovare il "cammino" che porta al risultato desiderato.

Il funzionamento di tali agenti **non** prevede l'uso di regole condizione-azione (tipiche degli agenti basati su modello), poiché considera il futuro in relazione a due domande: "Cosa accadrà se compio l'azione A?" e "Se faccio A, sarò soddisfatto?"

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
             stato, una descrizione dello stato corrente del mondo;
             goal, un obiettivo, inizialmente null;
             problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

Nota bene:

Mentre l'agente esegue la sequenza di azioni *ignora le proprie percezioni*, perché le conosce in anticipo

Un agente di questo tipo, che porta avanti le proprie azioni ad "occhi chiusi", deve essere certo di quello che accade.

Nella teoria del controllo, si parla di **sistema a ciclo aperto**, perché ignorando le percezioni si rompe il ciclo tra agente e ambiente.

Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

Un **problema** può essere definito formalmente da cinque componenti:

- **Stato iniziale.** Rappresenta il punto di partenza dell'agente. Nell'esempio precedente, lo stato iniziale potrebbe essere descritto come *in(Arad)*.

- **Azioni.** L'insieme *Azioni* include la descrizione delle possibili operazioni attuabili dall'agente. Più formalmente, dato uno stato *s*, *Azioni(s)* restituisce l'insieme di azioni che possono essere eseguite in *s*, anche dette azioni **applicabili** in *s*. Nell'esempio precedente, le azioni possibile sono {*Go(Sibiu)*, *Go(Timisoara)*, *Go(Zerind)*}.

- **Modello di transizione.** Descrive il risultato di ogni azione attuabile dall'agente in *s*. E' specificato da una funzione *Risultato(s, a)* che restituisce lo stato risultante dall'esecuzione dell'azione *a* nello stato *s*. Utilizziamo anche il termine **successore** per indicare qualsiasi stato raggiungibile da uno stato mediante una singola azione. Ad esempio, potremmo avere che: *Risultato(in(Arad), Go(Zerind)) = In(Zerind)*

- **Test obiettivo.** Questo determina se un particolare stato è uno stato obiettivo. In alcuni casi, esiste un insieme esplicito di possibili stati obiettivo: in questo caso il test si limiterà a verificare se uno stato *s* appartiene all'insieme. Nell'esempio, il test obiettivo sarà dato dal singoletto {*in(Bucarest)*}.

- **Costo di cammino.** Funzione che determina il costo numerico a ogni cammino. Nell'esempio, il costo potrebbe essere rappresentato dai chilometri da percorrere per arrivare a Bucarest.

Agenti Risolutori di Problemi

Formulazione di problemi

Oltre ad astrarre le descrizioni di stato, dobbiamo astrarre anche le azioni - un'azione di guida è influenzata e può avere molti effetti (ad esempio, la benzina viene consumata).

E' possibile essere più precisi nella definizione di un livello di astrazione *appropriato*?

Possiamo pensare ad uno stato non come un elemento individuale, ma come un **insieme di stati dettagliati**; allo stesso modo, un'azione è in realtà un **insieme di azioni detagliate**.

Un'astrazione si definisce **valida** se possiamo espandere ogni soluzione astratta in una soluzione nel mondo più detagliata.

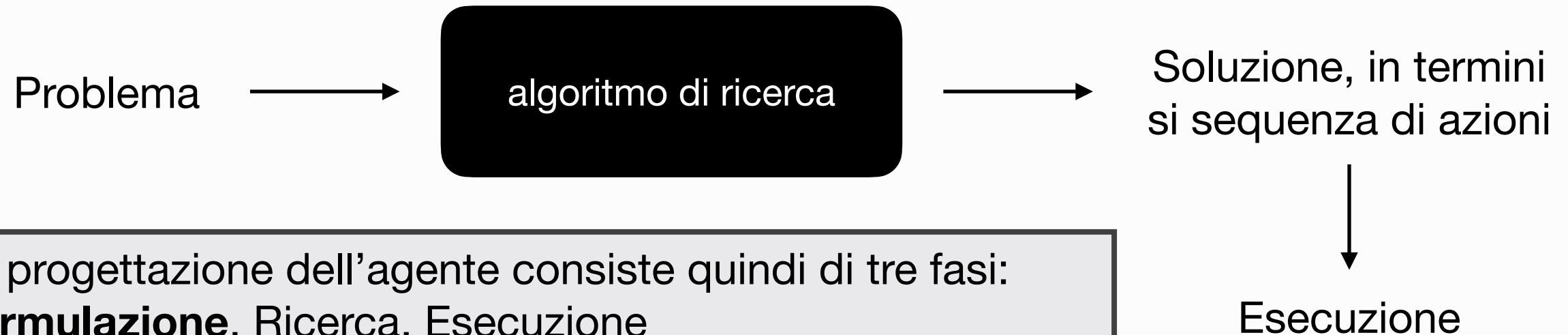
Un'astrazione si definisce **utile** se eseguire ogni azione nella soluzione è più facile che nel problema originale.

La scelta di una buona astrazione (o granularità dell'astrazione) prevede, quindi, che si rimuova quanto più dettaglio possibile mantenendo la validità e assicurandosi che le azioni astratte siano più facile da eseguire.

Senza la capacità di astrazione, *gli agenti intelligenti sarebbero completamente sovrachiati dalla complessità del mondo reale!*

Algoritmi di Ricerca Non Informata

Nella lezione precedente...



In particolare, ci siamo soffermati sulla parte di formulazione dei problemi di ricerca. In questa lezione, inizieremo a parlare di come poter effettivamente **ricercare soluzioni**.

Una soluzione è una **sequenza di azioni**. Per questo, gli algoritmi di ricerca operano considerando varie possibili di sequenze.

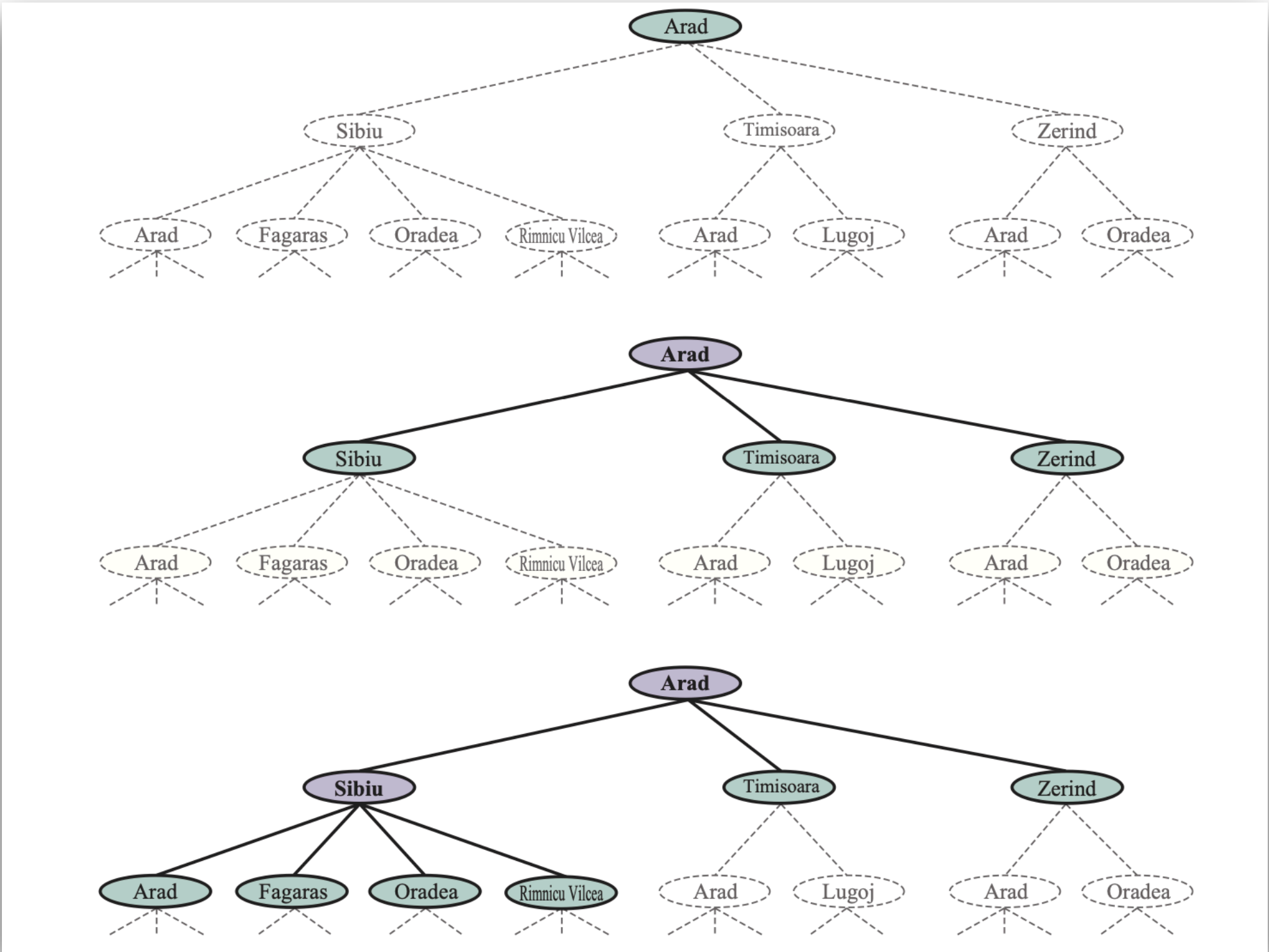
Tali sequenze di azioni formano un **albero di ricerca** con lo stato iniziale alla radice, i rami sono azioni e i nodi corrispondono a stati nello spazio degli stati del problema.

Dato lo stato iniziale, il primo passo sarà **verificare che l'obiettivo non sia stato raggiunto**; dopodiché, si considereranno le varie azioni, **espandendo** lo stato corrente e **generando** così un nuovo insieme di stati.

Questa è l'essenza della ricerca: **approfondire** un'opzione e mettere per il momento a parte le altre, pronti a riprenderle nel caso la prima non porti ad una soluzione.

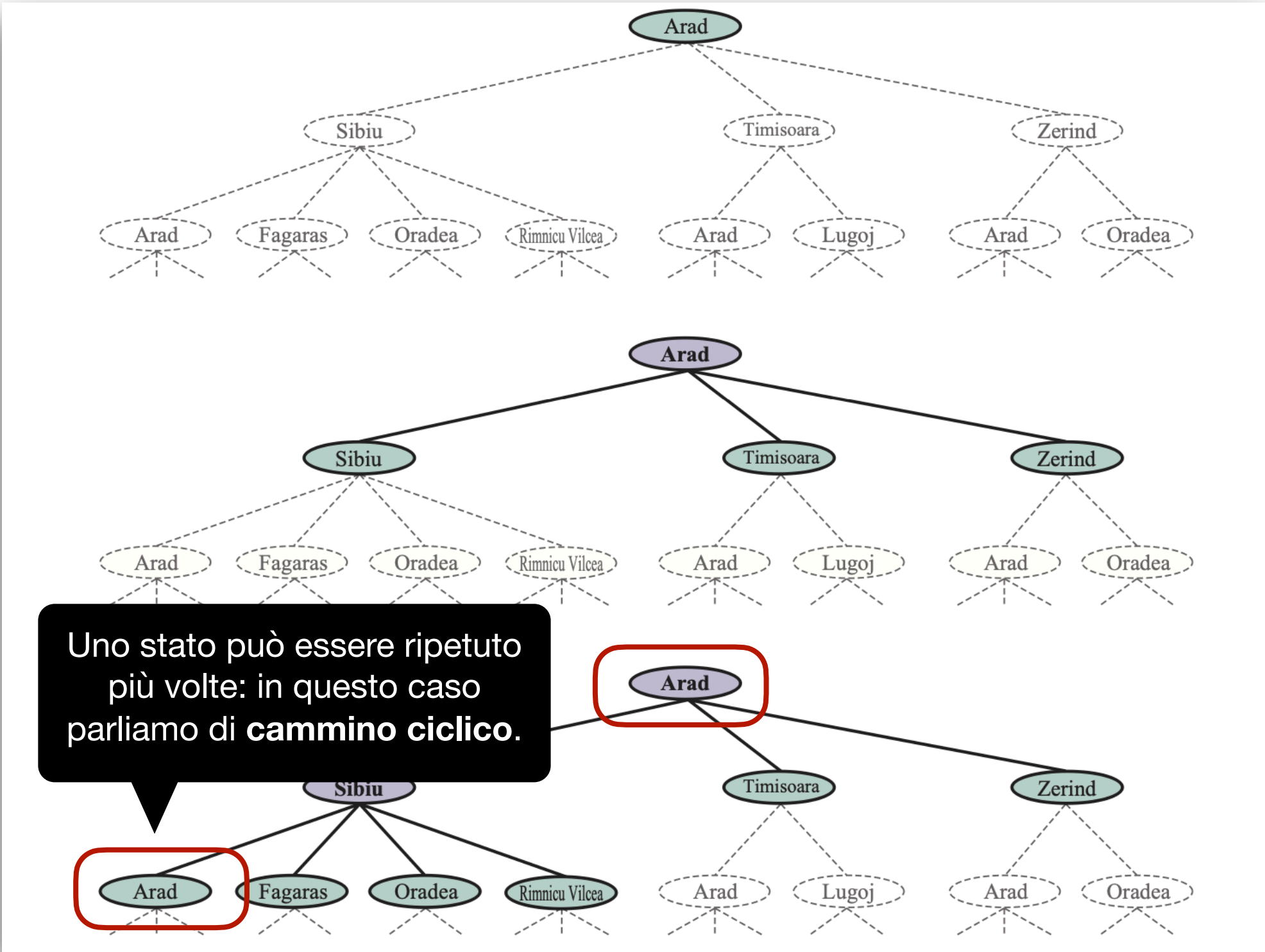
Algoritmi di Ricerca Non Informata

Cercare soluzioni



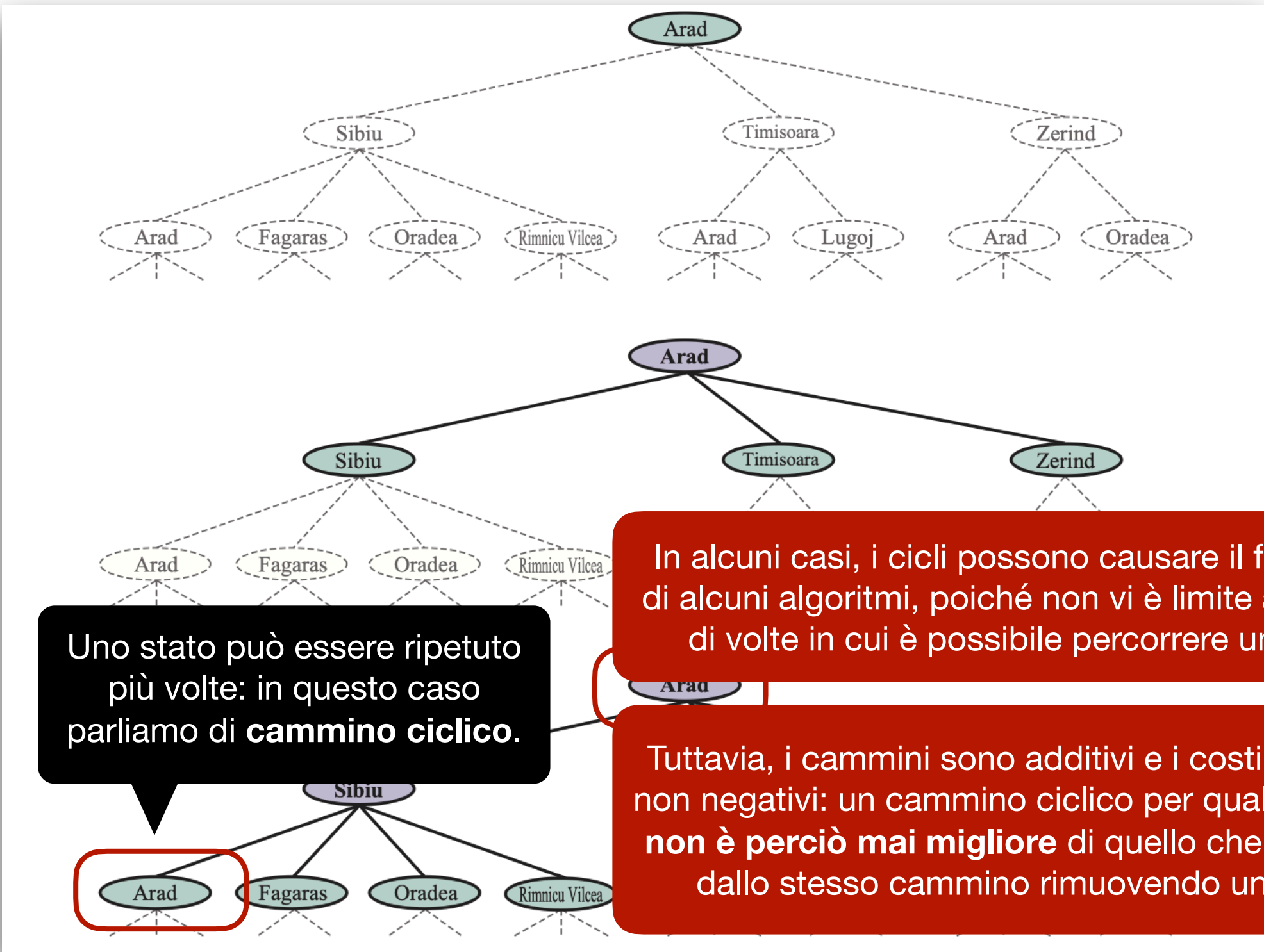
Algoritmi di Ricerca Non Informata

Cercare soluzioni



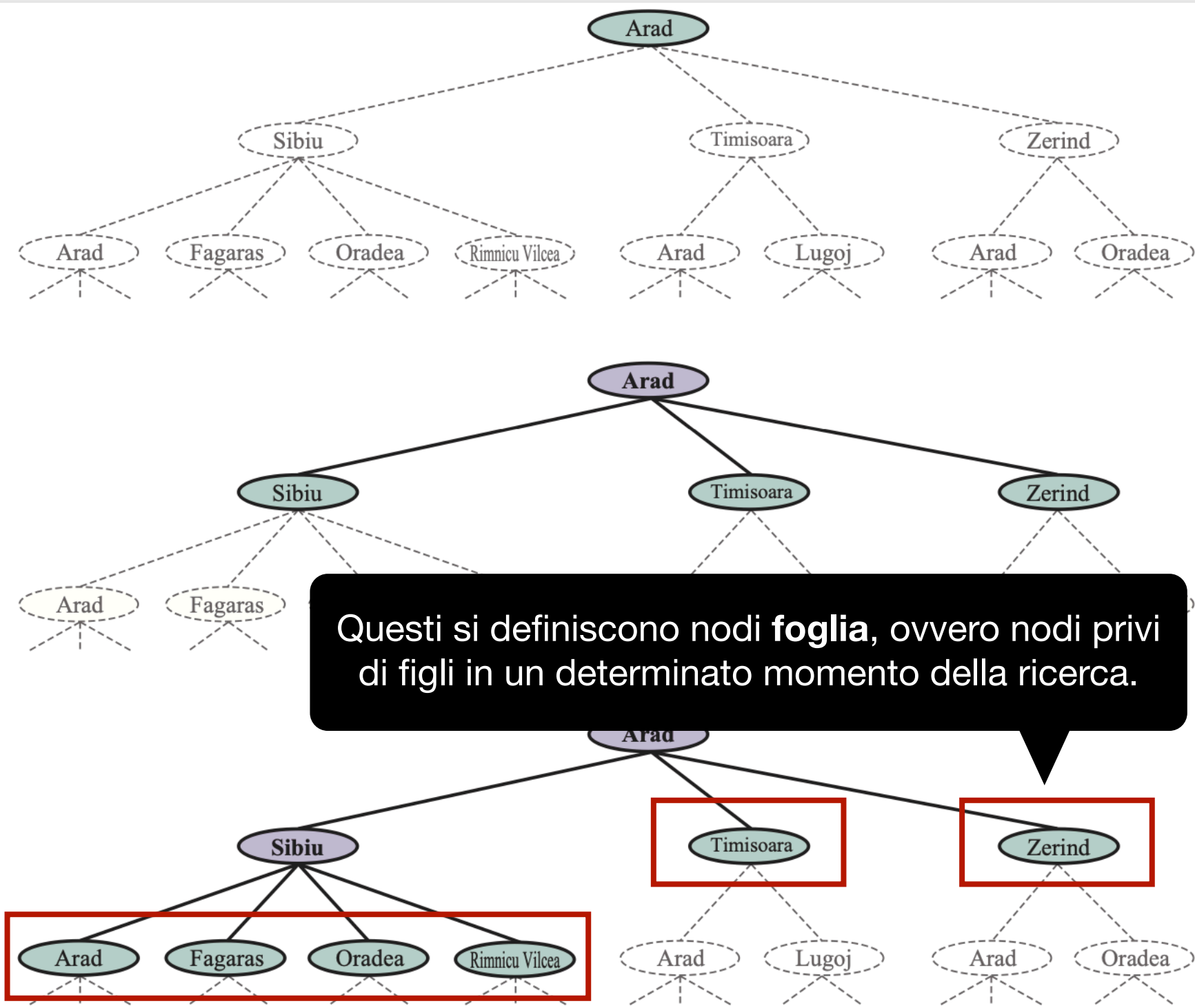
Algoritmi di Ricerca Non Informata

Cercare soluzioni



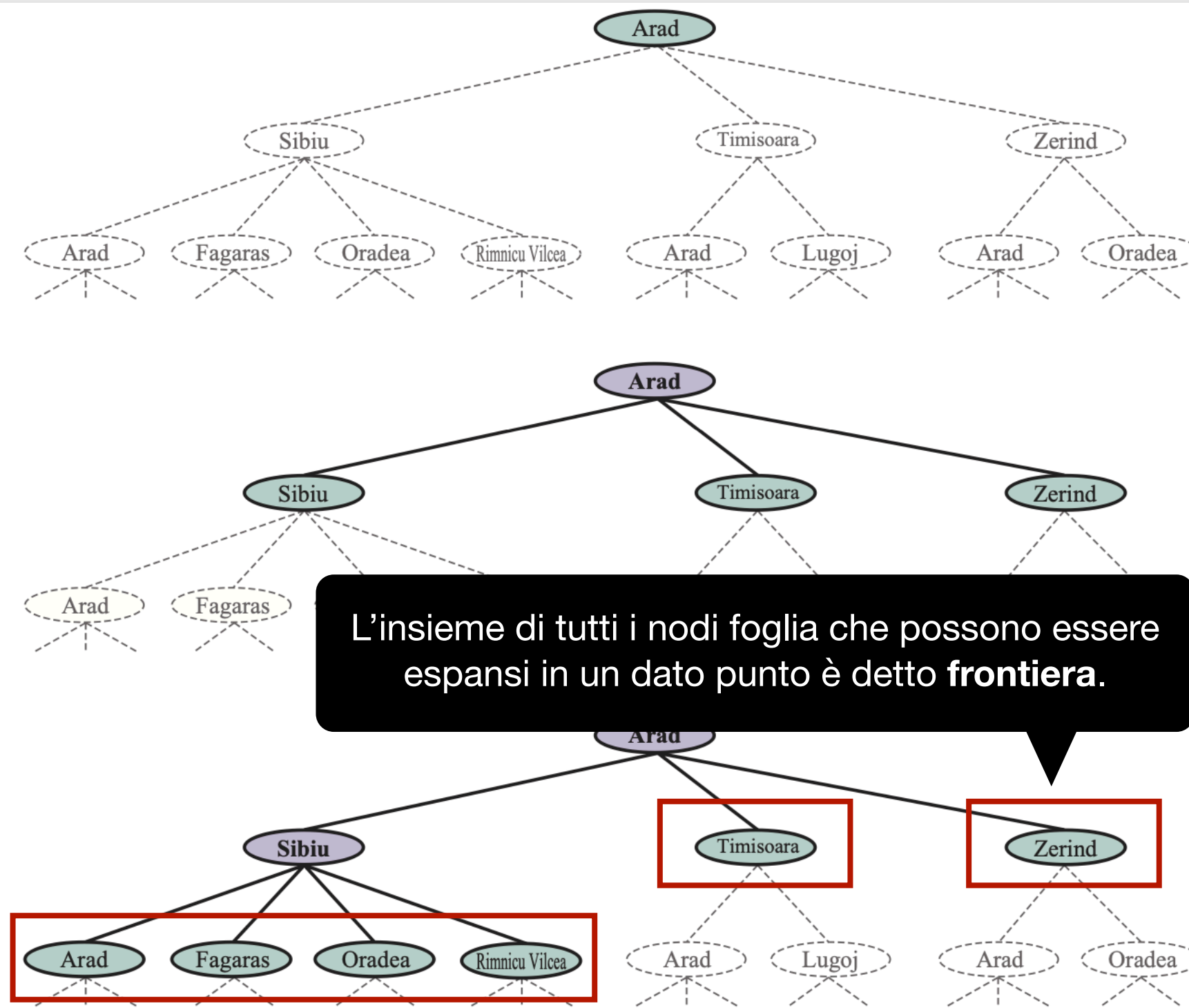
Algoritmi di Ricerca Non Informata

Cercare soluzioni



Algoritmi di Ricerca Non Informata

Cercare soluzioni



Algoritmi di Ricerca Non Informata

Il processo di espandere i nodi su una frontiera

```
function RICERCA-ALBERO(problema) returns una soluzione o un fallimento
  inizializza la frontiera usando lo stato iniziale di problema

  loop do

    if la frontiera è vuota then return fallimento
    scegli un nodo foglia e rimuovilo dalla frontiera

    if il nodo contiene uno stato obiettivo then return la soluzione corrispondente
    espandi il nodo scelto, aggiungendo i nodi risultati alla frontiera
```

Se volessimo essere più precisi, allora l'algoritmo dovrebbe prendere in considerazione una **strategia** in base alla quale poter espandere una frontiera:

```
function RICERCA-ALBERO(problema, strategia) returns una soluzione o un fallimento
  inizializza la frontiera usando lo stato iniziale di problema

  loop do

    if la frontiera è vuota then return fallimento
    scegli un nodo foglia in base a strategia e rimuovilo dalla frontiera

    if il nodo contiene uno stato obiettivo then return la soluzione corrispondente
    espandi il nodo scelto, aggiungendo i nodi risultati alla frontiera
```

Algoritmi di Ricerca Non Informata

Il processo di espandere i nodi su una frontiera

```
function RICERCA-ALBERO(problema) returns una soluzione o un fallimento
  inizializza la frontiera usando lo stato iniziale di problema

  loop do

    if la frontiera è vuota then return fallimento
    scegli un nodo foglia e rimuovilo dalla frontiera

    if il nodo contiene uno stato obiettivo then return la soluzione corrispondente
    espandi il nodo scelto, aggiungendo i nodi risultati alla frontiera
```

Se volessimo essere più precisi, allora l'algoritmo dovrebbe prendere in considerazione una **strategia** in base alla quale poter espandere una frontiera:

```
function RICERCA-ALBERO(problema, strategia) returns una soluzione o un fallimento
  inizializza la frontiera usando lo stato iniziale di problema

  loop do

    if la frontiera è vuota then return fallimento
    scegli un nodo foglia in base a strategia e rimuovilo dalla frontiera

    if il nodo contiene uno stato obiettivo then return la soluzione corrispondente
    espandi il nodo scelto, aggiungendo i nodi risultati alla frontiera
```

La **strategia** definisce la politica che l'algoritmo utilizzerà per decidere come procedere la ricerca.

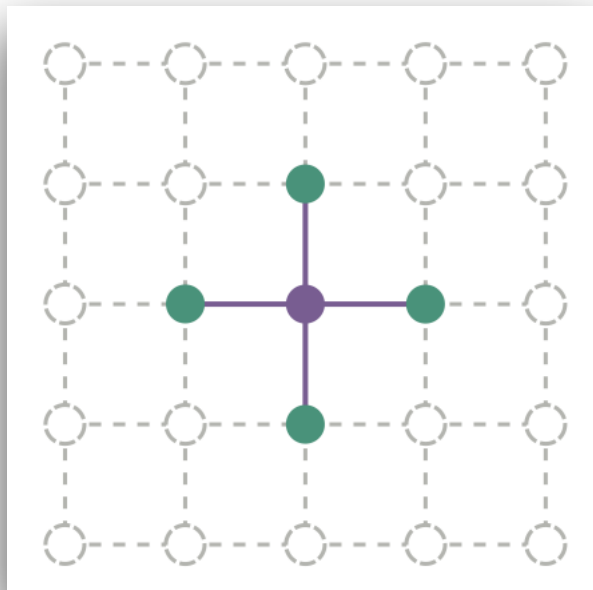
Algoritmi di Ricerca Non Informata

Cammini ciclici e cammini ridondanti

I cammini ciclici, ovvero quelli in cui uno stato è ripetuto nell'albero di ricerca, sono un caso particolare di **cammino ridondante**.

Cammino ridondante: Un cammino in cui esistono due o più modi per passare da uno stato all'altro.

Come detto, i cammini ridondanti possono talvolta causare il fallimento degli algoritmi di ricerca. In alcuni casi, tuttavia, non è possibile evitare i cammini ridondanti; pensate, ad esempio, ai problemi in cui le azioni sono *reversibili*.



Ricerca di un itinerario su griglia orizzontale: Ogni stato ha 4 successori, perciò l'albero di ricerca di profondità d che include stati ripetuti avrà 4^d foglie, nonostante ci siano “solo” $2 \cdot d^2$ stati distinti a d passi da qualsiasi stato dato.

Per $d = 20$, ciò significa che ci sono circa mille miliardi di nodi ma soltanto 800 stati distinti.

In altri termini, seguendo cammini ridondanti **si può trasformare un problema trattabile in uno intrattabile**.

Algoritmi di Ricerca Non Informata

Cammini ciclici e cammini ridondanti

Per evitare di addentrarsi in cammini ridondanti, è perciò necessario ricordare dove si è passati. Possiamo quindi migliorare l'algoritmo base di ricerca introducendo una struttura dati denominata **insieme esplorato** (o **lista chiusa**).

```
function RICERCA-GRAFO(problema, strategia) returns una soluzione o un fallimento
  inizializza la frontiera usando lo stato iniziale di problema
  inizializza a vuoto l'insieme esplorato

  loop do

    if la frontiera è vuota then return fallimento
    scegli un nodo foglia in base a strategia e rimuovilo dalla frontiera

    if il nodo contiene uno stato obiettivo then return la soluzione corrispondente
    aggiungi il nodo all'insieme esplorato
    espandi il nodo scelto, aggiungendo i nodi risultati alla frontiera solo se non è nella
                                                    frontiera o nell'insieme esplorato
```

L'albero di ricerca costruito con questo algoritmo contiene al più una copia di ciascuno stato, perciò possiamo pensare che faccia crescere un albero direttamente **come un grafo dello spazio degli stati**.

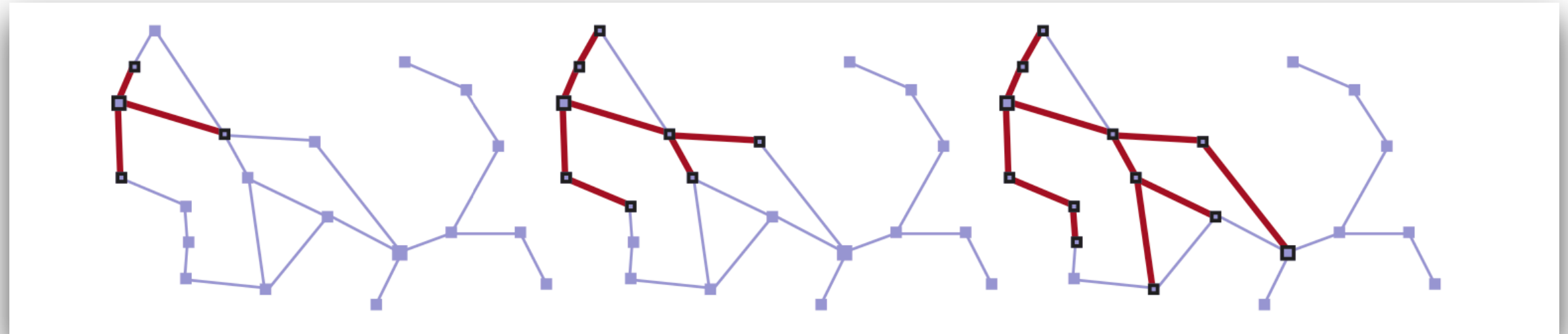
L'algoritmo ha un'altra proprietà: **la frontiera separa il grafo degli stati nella regione esplorata e in quella inesplorata**, in modo che ogni cammino che vado dallo stato iniziale ad uno stato inesplorato deve passare attraverso uno stato della frontiera.

Algoritmi di Ricerca Non Informata

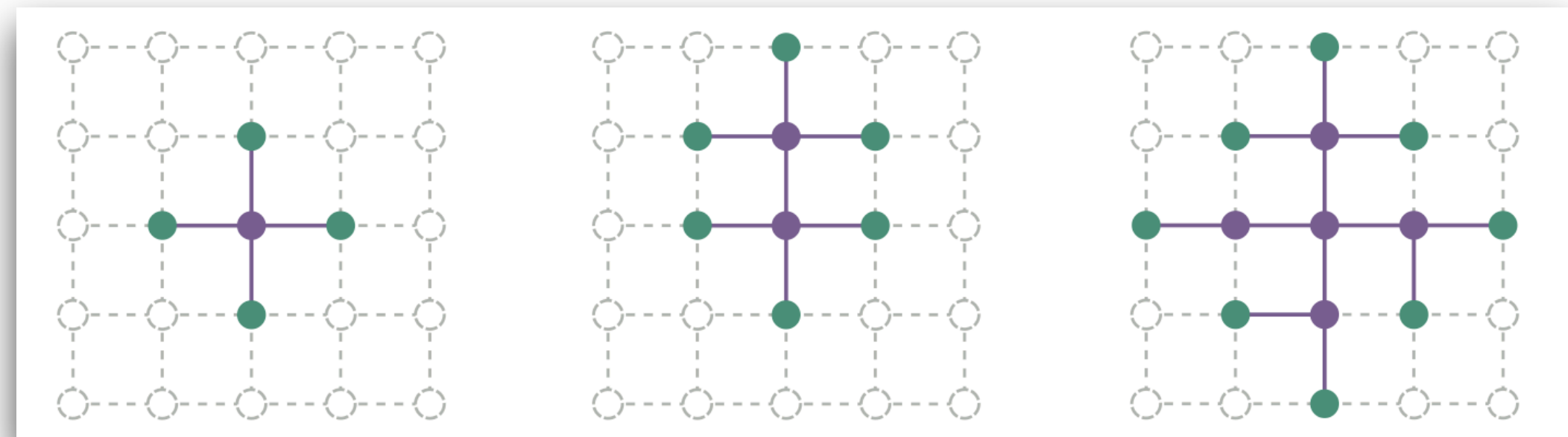
Cammini ciclici e cammini ridondanti

e si è

Proprietà 1



Proprietà 2



Algoritmi di Ricerca Non Informata

Strutture dati per algoritmi di ricerca

Gli algoritmi di ricerca richiedono una struttura dati per tenere traccia dell'albero di ricerca costruito. Per ogni nodo n dell'albero abbiamo una struttura contenente i seguenti quattro componenti:

- $n.stato$: lo stato dello spazio degli stati a cui corrisponde il nodo;
- $n.padre$: il nodo dell'albero di ricerca che ha generato il nodo corrente;
- $n.azione$: l'azione applicata al padre per generare il nodo;
- $n.costo-cammino$: il costo $g(n)$ del cammino che va dallo stato iniziale al nodo;

Dati i componenti di nodo padre, è poi facile calcolare i componenti necessari per generare un nodo figlio:

```
function NODO-FIGLIO(problema, padre, azione) returns un nodo
  returns un nodo con:

    STATO = problema.RISULTATO(padre.STATO, azione)
    PADRE = padre, AZIONE = azione

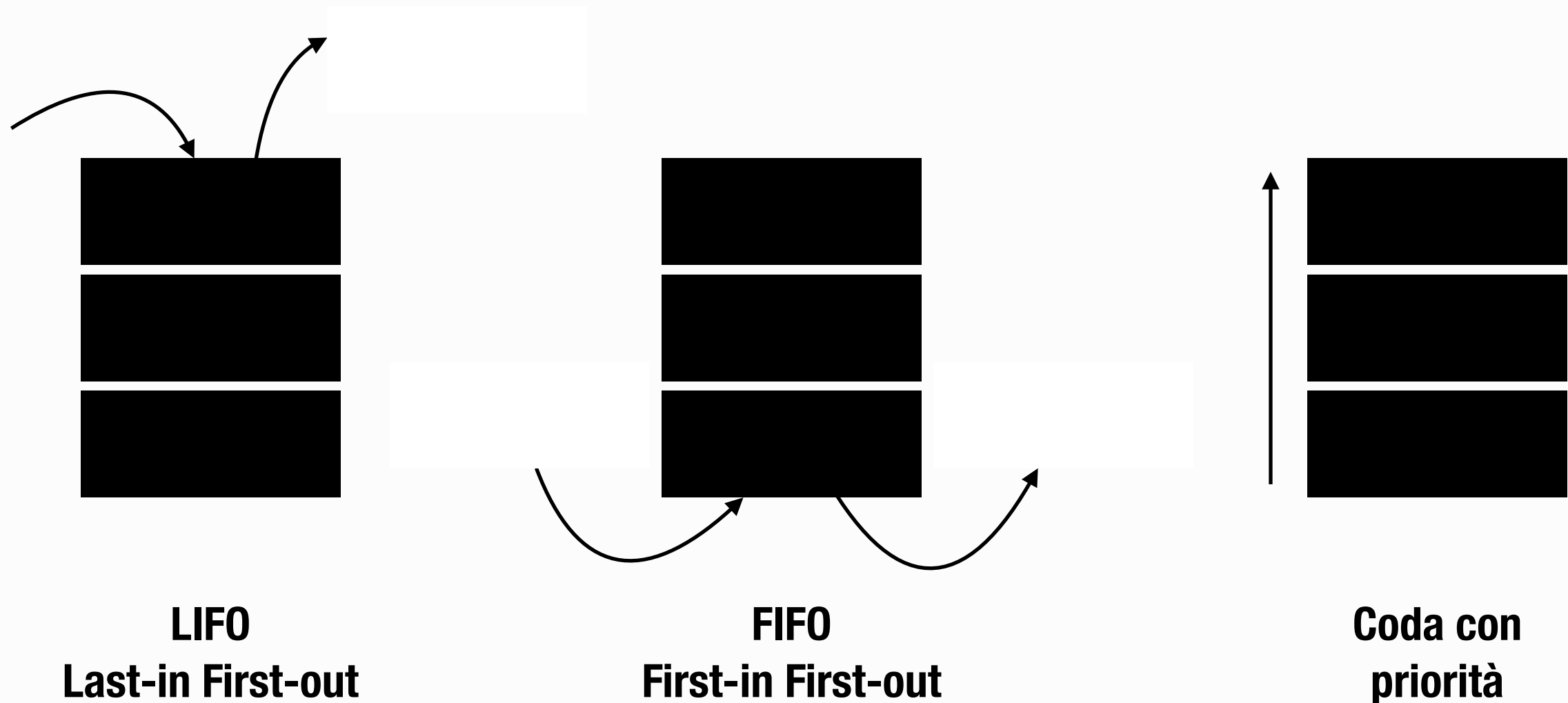
    COSTO-DI-CAMMINO = padre.COSTO-DI-CAMMINO + problema.COSTO-DI-PASSO(padre.STATO, azione, STATO)
```

Algoritmi di Ricerca Non Informata

Strutture dati per algoritmi di ricerca

Memorizzati i singoli nodi, è necessaria una struttura dove conservare tutti i nodi che saranno generati nell'albero di ricerca.

La frontiera deve essere memorizzata in modo da consentire all'algoritmo di ricerca di scegliere facilmente il successivo nodo da espandere in base alla propria strategia. In tal senso, la struttura dati più adatta è una **coda**.



Algoritmi di Ricerca Non Informata

Valutare un algoritmo di ricerca

Una strategia di ricerca è definita secondo una **modalità di espansione dei nodi**, ovvero scegliendo l'ordine in cui i nodi sono espansi.

Tali strategie vengono valutate in base a quattro indicatori:

Completezza: L'algoritmo *garantisce* di trovare una soluzione, se questa esiste?

Ottimalità: L'algoritmo garantisce di trovare la *soluzione ottima*?

Complessità temporale: Quanto tempo *impiega* l'algoritmo per trovare una soluzione?

Complessità spaziale: Di quanta *memoria* ha bisogno l'algoritmo per trovare una soluzione?

A differenza dell'informatica teorica, dove la complessità spaziale e temporale sono tipicamente misurate in termini di dimensione del grafo (ovvero, in termini di $|V|$ e $|E|$) poiché la struttura dati viene esplicitamente passata come input di un algoritmo, in Intelligenza Artificiale il grafo è spesso *implicitamente* rappresentato dallo stato iniziale, dalle azioni e dal modello di transizione: per questa ragione, è spesso *indefinito*.

Ne consegue che la complessità viene espressa in termini diversi: parliamo di (i) b , il fattore di ramificazione o branching factor, (ii) d , profondità della soluzione a costo minimo o depth, e (iii) m , massima profondità dello spazio degli stati.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata

Gli algoritmi di ricerca non informata fanno riferimento alle strategie di ricerca che **non dispongono di informazioni aggiuntive sugli stati** oltre a quella fornita nella definizione del problema: tutto ciò che possono fare è generare successori e distinguere gli stati obiettivo dagli altri.

La principale differenza tra le varie strategie di ricerca consiste nell'**ordine in cui vengono espansi i nodi**.

Data la loro natura, gli algoritmi di ricerca non informata **non sanno stimare quanto un nodo non obiettivo sia “promettente” per la risoluzione del problema**, a differenza degli algoritmi di ricerca informata e euristica, i quali fanno uso di informazioni riguardo la distanza stimata dalla soluzione.

Nel contesto di questo corso, ci soffermeremo su:

- Ricerca in ampiezza;
- Ricerca di costo uniforme;
- Ricerca in profondità;
- Ricerca in profondità limitata;
- Ricerca con approfondimento iterativo;
- Ricerca bidirezionale.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

La ricerca in ampiezza è una semplice strategia che consiste di:

1. Espandere il nodo radice;
2. Espandere i nodi generati dalla radice;
3. Espandere i loro successori e così via.

Ricerca in ampiezza: Strategia di ricerca in cui tutti i nodi di profondità d sono espansi prima di quelli di profondità $d+1$.

La ricerca in ampiezza è una strategia **sistematica** di ricerca, ovvero è in grado di trovare sempre una soluzione, se esiste. E' inoltre, di trovare sempre la soluzione realizzabile con il **cammino più breve**: tuttavia, bisogna sottolineare che il cammino più breve non è necessariamente quello con il costo minore.

La ricerca in ampiezza è un'istanza dell'algoritmo generale di ricerca su grafo in cui **viene scelto per l'espansione il nodo non espanso più vicino alla radice**.

Da un punto di vista pratico, questa strategia può essere implementata utilizzando una semplice **coda FIFO per la frontiera**. Di conseguenza, i nuovi nodi (che sono quelli via via più lontani dalla radice) vanno in fondo alla coda e i nodi vecchi (che sono quelli più vicini) vengono espansi per primi.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← {}

loop do
    /* All'inizio, non facciamo altro che assegnare la
       radice del grafo alla variabile nodo e controllare se la
       soluzione è la radice */

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop
    /* Se la soluzione non è nella radice, allora
    inizializziamo la frontiera (con la radice, che è l'unico
    nodo esplorato finora) e la lista dei nodi esplorati */

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    aggiungi nodo.STATO a esplorati

    for figlio in generati(nodo) do
        /* Se, durante la ricerca, la frontiera dovesse essere vuota e la soluzione non dovesse essere trovata, allora non resta che restituire un fallimento */
        if figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← nodo.STATO.azione(azione)
        if figlio non è in frontiera then
            aggiungi figlio a frontiera
            if figlio.STATO in esplorati then return SOLUZIONE(figlio)
            else
                /* La funzione POP di una coda estrae il primo elemento
                della frontiera, di fatto procedendo alla sua espansione;
                il nuovo nodo verrà aggiunto a quelli esplorati */
                aggiungi figlio a esplorati
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza

```
function RICERCA-IN-AMPIEZZA(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop do

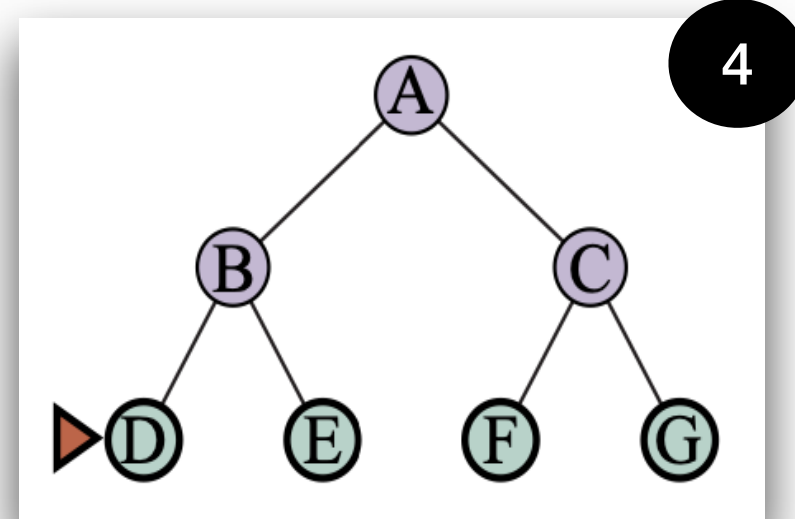
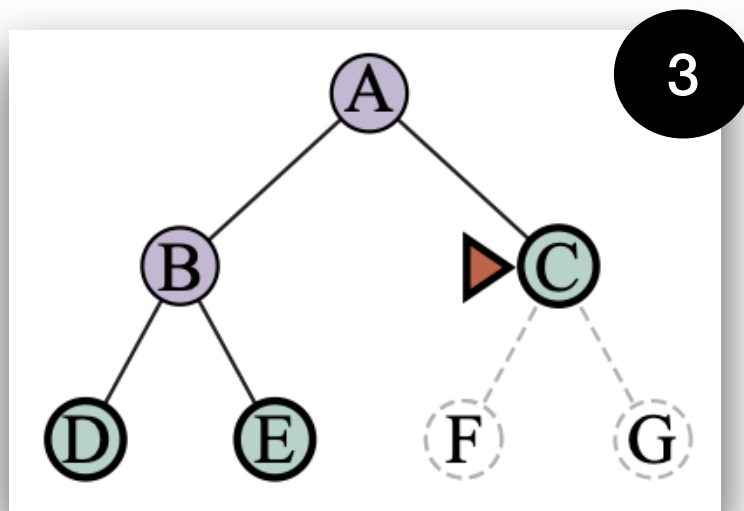
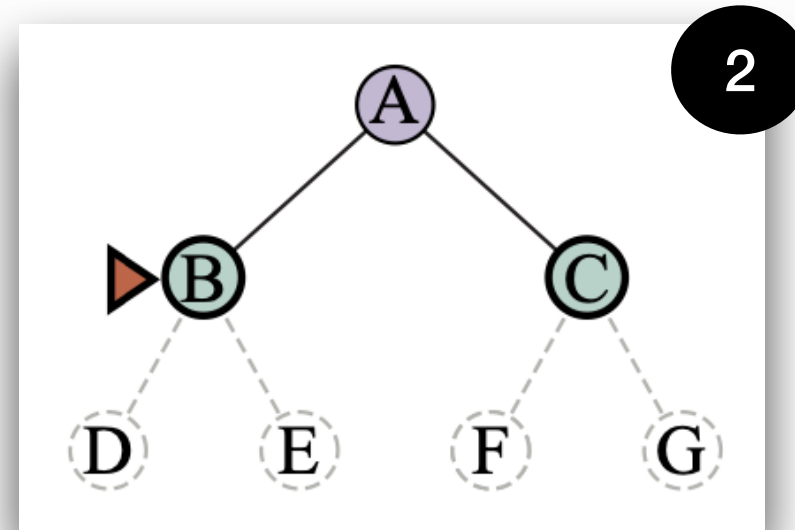
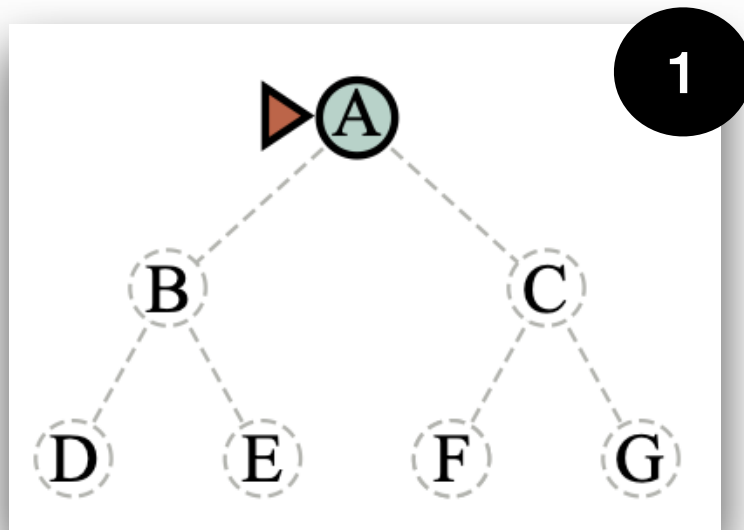
    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
```

/* La ricerca va avanti: Dal nodo considerato si procede ad esplorare i figli (solo se non già esplorati) e valutare se questi rappresentano delle soluzioni. */

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza, un esempio pratico



Ad ogni iterazione, viene espanso il nodo indicato. Si può pensare alla ricerca in ampiezza come un pendolo che, di volta in volta, visita i nodi in maniera orizzontale.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza, performance

Analizziamo adesso le prestazioni della ricerca in ampiezza, considerando i quattro indicatori precedentemente introdotti.

Denotiamo con b il fattore di diramazione (ovvero, il numero massimo di successori); e con d la profondità della soluzione minima.

Completezza: E' facilmente intuibile che l'algoritmo è completo. Se il nodo obiettivo più vicino alla radice si trova alla profondità d , la ricerca lo troverà dopo aver espanso tutti i nodi che lo precedono.

Ottimalità: Se il costo di cammino è una funzione monotona non decrescente della profondità del nodo, allora possiamo dire che la ricerca è ottimo. Più in generale, però, l'algoritmo non lo è - restituisce la soluzione più vicina, indipendentemente dal costo.

Complessità temporale: Supponiamo di dover ricercare una soluzione in un albero in cui ogni nodo ha b successori. La radice genererà b nodi al primo livello, ognuno dei quali genererà altri b nodi, per un totale di b^2 al secondo livello. Ognuno di questi genererà altri b nodi, arrivando a b^3 al terzo livello, e così via. Nel caso pessimo, la soluzione è l'ultima ad essere analizzata e si trova ad una profondità d . Quindi:

$$b + b^2 + b^3 + \dots + b^d = O(b^d)$$

La complessità temporale è esponenziale

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza, performance

Analizziamo adesso le prestazioni della ricerca in ampiezza, considerando i quattro indicatori precedentemente introdotti.

Denotiamo con b il fattore di diramazione (ovvero, il numero massimo di successori); e con d la profondità della soluzione minima.

Complessità temporale: Supponiamo di dover ricercare una soluzione in un albero in cui ogni nodo ha b successori. La radice genererà b nodi al primo livello, ognuno dei quali genererà altri b nodi, per un totale di b^2 al secondo livello. Ognuno di questi genererà altri b nodi, arrivando a b^3 al terzo livello, e così via. Nel caso pessimo, la soluzione è l'ultima ad essere analizzata e si trova ad una profondità d . Quindi:

$$b + b^2 + b^3 + \dots + b^d = O(b^d)$$

La complessità temporale è esponenziale

Complessità spaziale: L'algoritmo memorizza ogni nodo espanso nell'insieme *esplorati*: per questa ragione, la complessità sarà sempre una funzione di b . In particolare, la ricerca in ampiezza conserva *ogni* nodo generato in memoria: avremo così $O(b^{d-1})$ nodi nell'insieme esplorato e $O(b^d)$ nodi nella frontiera. Quindi, la complessità spaziale sarà di $O(b^d)$, ovvero è dominata dalla dimensione della frontiera.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in ampiezza, complessità

Che cosa implica una tale complessità in pratica?

Profondità	Nodi	Tempo	Memoria
2	1.100	0,11 sec	1 Mb
4	111.000	11 sec	106 Mb
6	10 ⁷	19 min	10 Gb
8	10 ⁹	31 ore	1 Tb
10	10 ¹¹	129 giorni	101 Tb
12	10 ¹³	35 anni	10 Pb
14	10 ¹⁵	3.523 anni	1 Eb

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

La ricerca a costo uniforme rappresenta una variante della ricerca in ampiezza che, come vedremo, è capace di trovare sempre la soluzione ottima (e non solo quando il costo di passo è una funzione monotona).

Ricerca a costo uniforme: Piuttosto che espandere il nodo meno profondo, la ricerca a costo uniforme espande il nodo n con il minimo costo di cammino $g(n)$.

Semplicemente, questo è possibile memorizzando la frontiera come **coda a priorità ordinata secondo g** - in cima alla coda ci saranno i nodi di costo minore.

Oltre all'ordinamento della coda, ci sono due sostanziali differenze implementative rispetto alla ricerca per ampiezza:

1. Il test obiettivo è applicato ad un nodo non appena è *selezionato* per l'espansione - e non quando è *generato* per la prima volta.
2. Si aggiunge un test obiettivo nel caso in cui *sia trovato un cammino migliore* per raggiungere un nodo che attualmente si trova sulla frontiera.

Cerchiamo di capire il motivo alla base delle due differenze. Nella slide successiva, sono riportate in **rosso** le linee di codice appartenenti all'algoritmo di ricerca per ampiezza che sono rimosse dall'algoritmo a costo uniforme; in **verde**, invece, le linee che implementano le differenze con l'algoritmo di ricerca per ampiezza.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

```
function RICERCA-IN-AMPIEZZA RICERCA-COSTO-UNIFORME(problema, strategia)
    returns una soluzione o un fallimento

    nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
    frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
    esplorati ← un insieme vuoto

    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

    frontiera ← una coda FIFO con nodo come unico elemento
    esplorati ← un insieme vuoto

    loop do

        if la frontiera è vuota then return fallimento
        nodo ← POP(frontiera)
        if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
        aggiungi nodo.STATO a esplorati

        for each azione in problema.AZIONI(nodo.STATO) do
            figlio ← NODO-FIGLIO(problema, stato, azione)
            if figlio.STATO non è in esplorati e figlio non è in frontiera then
                if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
                frontiera ← INSERISCI(figlio, frontiera)
            else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
                sostituisci quel nodo frontiera con figlio
```


Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

```
function RICERCA-IN-AMPIEZZA RICERCA-COSTO-UNIFORME(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
esplorati ← un insieme vuoto

if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← vuoto
/* Se il test fosse fatto a questo punto, rischieremmo di ritornare
   una soluzione sub-ottima, poiché il primo nodo generato
   potrebbe trovarsi su un cammino con costo maggiore. */

loop do
    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
            sostituisci quel nodo frontiera con figlio
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

```
function RICERCA-IN-AMPIEZZA RICERCA-COSTO-UNIFORME(problema, strategia)
    returns una soluzione o un fallimento

    nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
    frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
    esplorati ← un insieme vuoto

    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

    frontiera ← una coda FIFO con nodo come unico elemento
    esplorati ← un insieme

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
            sostituisci quel nodo frontiera con figlio
```

/* Ricordiamo che la funzione POP di una coda a priorità restituisce il nodo di costo minimo in frontiera: pertanto, se il test ha successo, allora abbiamo trovato la soluzione migliore. */

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

```
function RICERCA-IN-AMPIEZZA RICERCA-COSTO-UNIFORME(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
esplorati ← un insieme vuoto

if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati
    /* Per la stessa ragione, non effettuiamo il test qui perché il figlio
       potrebbe trovarsi su un cammino sub-ottimo. */
    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(nodo.STATO, problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
            frontiera ← INSERISCI(figlio, frontiera)
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
            sostituisci quel nodo frontiera con figlio
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme

```
function RICERCA-IN-AMPIEZZA RICERCA-COSTO-UNIFORME(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
esplorati ← un insieme vuoto

if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)

frontiera ← una coda FIFO con nodo come unico elemento
esplorati ← un insieme vuoto

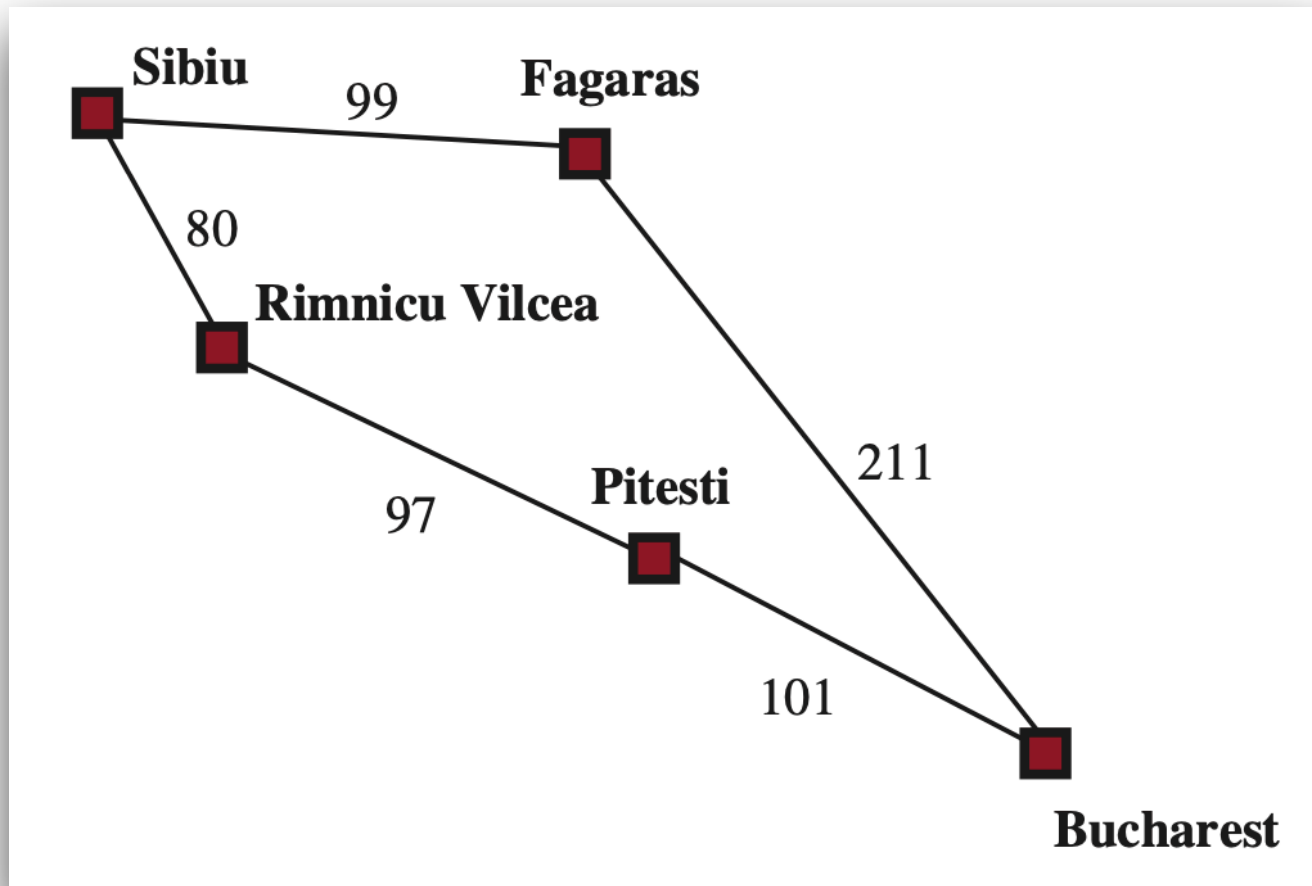
loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati

    for
        /* L'ultimo if consente di modificare l'ordinamento della lista e di
           conseguenza l'esplorazione dei nodi - in questo modo, si potrà
           disporre sempre dei nodi in ordine crescente di costo. */
        if problema.TEST-OBIETTIVO(figlio.STATO) then return SOLUZIONE(figlio)
        frontiera ← INSERISCI(figlio, frontiera)
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
            sostituisci quel nodo frontiera con figlio
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme, un esempio pratico



Obiettivo: Raggiungere Bucarest da Sibiu.

I successori di Sibiu sono Vilcea e Fagaras, che distano 80 e 99 km, rispettivamente, da Sibiu.

Viene espanso il nodo con costo minore, Vilcea. A questo punto, viene generato il nodo figlio, Pitesti.

Il costo di cammino sarà quindi pari a $80 + 97 = 177$.

Ora il nodo con costo minore sarà Fagaras, con 99 (< 177). La ricerca procederà quindi con la sua espansione verso Bucarest: il costo di cammino sarà di $99 + 211 = 310$.

Sebbene sia stato raggiunto l'obiettivo, la ricerca a costo uniforme continua poiché esiste un nodo con costo minore di 310. Il nodo Pitesti verrà espanso, il che porterà al raggiungimento di Bucarest con costo $177 + 101 = 288$.

L'algoritmo verificherà che questo cammino è migliore del precedente ($288 < 310$) e restituirà, quindi, la soluzione finale.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca a costo uniforme, performance

Da un punto di vista di performance, la ricerca a costo uniforme:

Completezza e ottimalità: E' nuovamente facilmente intuibile che l'algoritmo è completo (a meno di operazioni NoOp). Inoltre, per natura, l'algoritmo è anche ottimo.

Complessità temporale e spaziale: La ricerca a costo uniforme è guidata dal costo di cammino e non dalla loro profondità, per questo non è semplice definire la sua complessità in termini di fattore di ramificazione b e profondità d .

Consideriamo perciò C^* , il costo della soluzione ottima e assumiamo che ogni azioni costi almeno ϵ . La complessità temporale e spaziale sarà uguale a $O(b^{1 + \lfloor C^* / \epsilon \rfloor})$.

La spiegazione è molto semplice: C^* / ϵ rappresenta il costo di ogni passo fatto dall'algoritmo (arrotondato per difetto); in maniera simile alla ricerca in ampiezza, la complessità è dominata dal numero di passi effettuati, con l'aggravante che non si arresta una volta raggiunto l'obiettivo, ma esamina tutti i nodi al livello di profondità dell'obiettivo per verificare l'esistenza di una soluzione con costo minore.

Quindi, la ricerca a costo uniforme è, in generale, più efficace di quella in ampiezza ma potrebbe essere addirittura meno efficiente!

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità

Come il nome suggerisce, questo tipo di ricerca è esattamente l'opposto della ricerca in ampiezza; di fatti, non espande i nodi come un pendolo ma procede analizzando un ramo alla volta dell'albero di ricerca.

Ricerca in profondità: Strategia di ricerca in cui viene sempre espanso prima il nodo più profondo nella frontiera corrente dell'albero di ricerca.

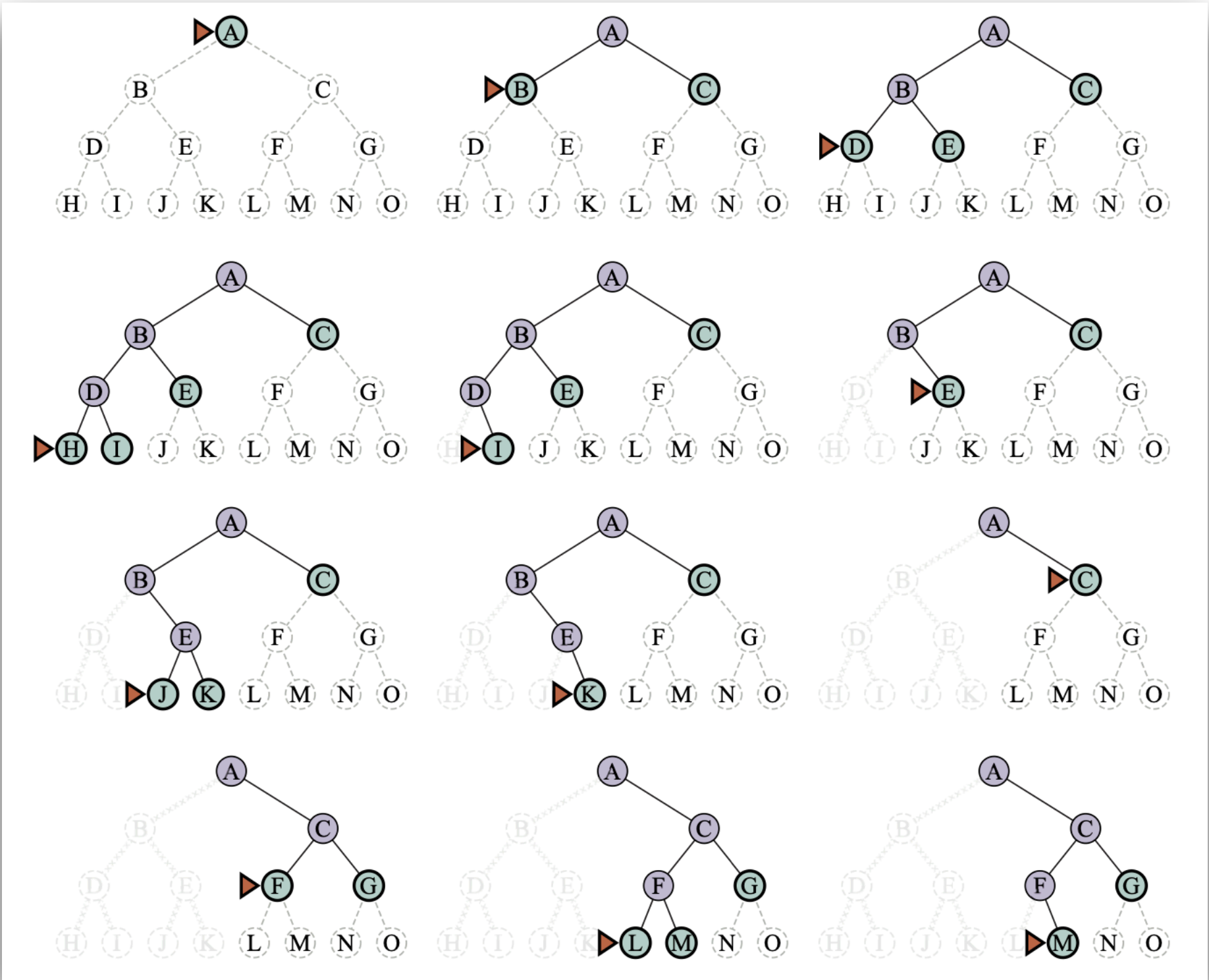
Da un punto di vista pratico, la ricerca in profondità raggiunge immediatamente il livello più profondo dell'albero, dove i nodi non hanno successori. L'espansione di tali nodi li rimuove dalla frontiera, per cui la ricerca “torna indietro” (*backtracking*, un termine che incontreremo di nuovo in futuro) per riconsiderare il nodo più profondo che ha successori non ancora espansi.

Questo tipo di ricerca fa uso di una coda LIFO, poiché verrà sempre espanso l'ultimo nodo generato.

Nello studio dello pseudocodice che implementa la ricerca, considereremo il caso in cui l'algoritmo sia *ricorsivo*: in questo contesto, un algoritmo è formulato in maniera da essere funzione di sé stesso, ovvero capace di eseguire sé stesso più volte con diversi parametri di input.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità, un esempio pratico



Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità, un esempio pratico

Qual è il problema?

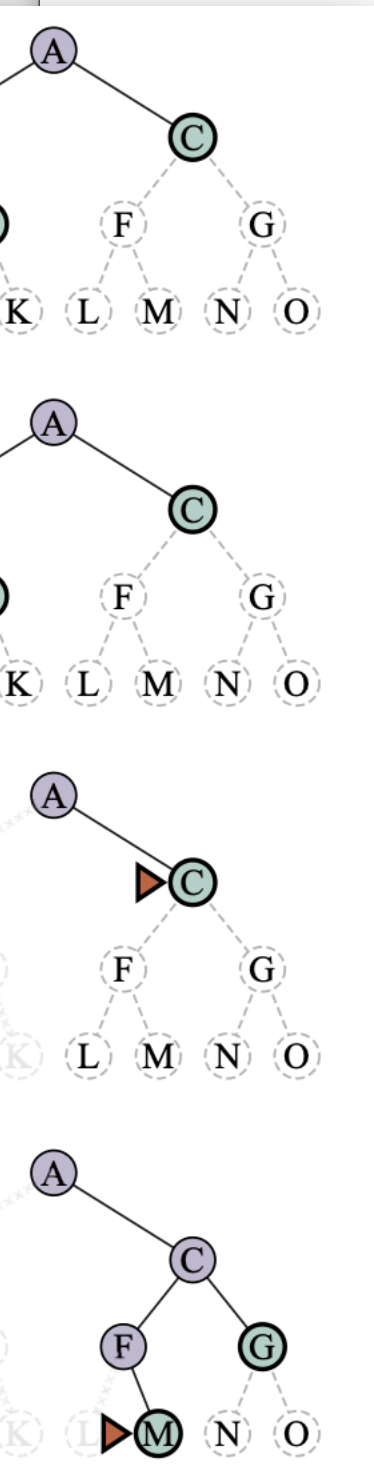
Cammino ridondante: Un cammino in cui esistono due o più modi per passare da uno stato all'altro.

In casi in cui l'albero di ricerca ha uno spazio degli stati con profondità infinita o con cicli, l'algoritmo non terminerà. Quindi, possiamo sostenere che l'algoritmo di ricerca in profondità **non è completo**.

Allo stesso tempo, la ricerca in profondità può ritornare un nodo obiettivo lontano dalla radice (andando, appunto, in profondità), perdendo contatto con l'ottimo globale. Quindi, l'algoritmo **non è ottimo**.

Sebbene abbia dei vantaggi in termini di complessità spaziale rispetto agli algoritmi visti precedentemente, la complessità temporale resta esponenziale. Quindi, di per sé la ricerca in profondità non è interessante.

Possiamo però migliorare l'algoritmo, almeno dal punto di vista della completezza, imponendo un limite massimo alla profondità dei cammini.



Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

Opera in maniera simile alla strategia di ricerca in profondità ma impone un limite di profondità per evitare che l'algoritmo non termini.

Ricerca in profondità limitata: Con questa strategia, un nodo viene espanso solo se la lunghezza del cammino corrispondente è minore rispetto al massimo stabilito.

Da un lato, l'algoritmo evita la non terminazione; dall'altro, però, introduce una nuova fonte di incompletezza nel caso in cui la soluzione si trovi ad una profondità maggiore di quella stabilita.

Rispetto ai precedenti algoritmi, questo tipo di ricerca può terminare in due modi diversi: o per via di un fallimento (ovvero, nessuna soluzione) o tramite il valore speciale *taglio* (ovvero, nessuna soluzione entro il limite stabilito).

Il problema di questa strategia consiste nel selezionare una soglia tale da poter garantirne il successo. Talvolta, questa soglia deriva dall'esperienza del designer o da dati empirici; altre volte, invece, non è possibile definirla.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

```
function RICERCA-PROFONDITÀ-LIMITATA(problema, strategia)
    returns una soluzione o il fallimento/taglio

returns RPL-RICORSIVA(CREA-NODO(problema.STATO-INIZIALE), problema, limite)
```

```
function RPL-RICORSIVA(nodo, problema, limite)
    returns una soluzione o il fallimento/taglio

if problema.TEST-OBIETTIVO(nodo.STATO) then returns SOLUZIONE(nodo)
else if limite = 0 then return taglio

else
    avvenuto_taglio ← false
    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, nodo, azione)
        risultato ← RPL-RICORSIVA(figlio, problema, limite-1)
        if risultato = taglio then avvenuto_taglio ← true
        else if risultato != fallimento then return risultato

if avvenuto_taglio then return taglio
else return fallimento
```

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

```
function RICERCA-PROFONDITÀ-LIMITATA(problema, strategia)
    returns una soluzione o il fallimento/taglio

returns RPL-RICORSIVA(CREA-NODO(problema.STATO-INIZIALE), problema, limite)
```

```
function RPL-RICORSIVA(nodo, problema, limite)
    io
    /* Il return statement della funzione madre restituirà il risultato/
       fallimento fornito dalla funzione ricorsiva */
    if problema
    else if limite = 0 then return taglio
    nodo)

    else
        avvenuto_taglio <- false
        for each azione in problema.AZIONI(nodo.STATO) do
            figlio <- NODO-FIGLIO(problema, nodo, azione)
            risultato <- RPL-RICORSIVA(figlio, problema, limite-1)
            if risultato = taglio then avvenuto_taglio <- true
            else if risultato != fallimento then return risultato

    if avvenuto_taglio then return taglio
    else return fallimento
```


Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

```
function RICERCA-PROFONDITÀ-LIMITATA(problema, strategia)
    returns una soluzione o il fallimento/taglio

returns RPL-RICORSIVA(CREA-NODO(problema.STATO-INIZIALE), problema, limite)
```

```
function RPL-RICORSIVA(nodo, problema, limite)
    returns una soluzione o il fallimento/taglio

    if problema.TEST-OBIETTIVO(nodo.STATO) then returns SOLUZIONE(nodo)
    else if limite = 0 then return taglio

    else
        avvenuto_taglio ← false
        for each figlio in nodo.genera_figli()
            risultato ← RICERCA-PROFONDITÀ-LIMITATA(figlio, problema, limite)
            if risultato = taglio then avvenuto_taglio ← true
            else if risultato != fallimento then return risultato

        if avvenuto_taglio then return taglio
        else return fallimento
```

/* Il limite servirà a terminare la ricerca ed evitare possibili casi di incompletezza */

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

```
function RICERCA-PROFONDITÀ-LIMITATA(problema, strategia)
    returns una soluzione o il fallimento/taglio

returns RPL-RICORSIVA(CREA-NODO(problema.STATO-INIZIALE), problema, limite)
```

```
function RPL-RICORSIVA(nodo, problema, limite)
    returns una soluzione o il fallimento/taglio

if problema.TEST-OBIETTIVO(nodo.STATO) then
else if limite = 0 then return taglio

else
    avvenuto_taglio <- false
    for each azione in problema.AZIONI(nodo.STATO) do
        figlio <- NODO-FIGLIO(problema, nodo, azione)
        risultato <- RPL-RICORSIVA(figlio, problema, limite-1)
        if risultato = taglio then avvenuto_taglio <- true
        else if risultato != fallimento then return risultato

if avvenuto_taglio then return taglio
else return fallimento
```

/* Il processo di ricerca è pressoché identico all'algoritmo generale di ricerca, con l'eccezione del fatto che la funzione sarà ricorsiva */

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata, performance

Da un punto di vista di performance, la ricerca a profondità limitata:

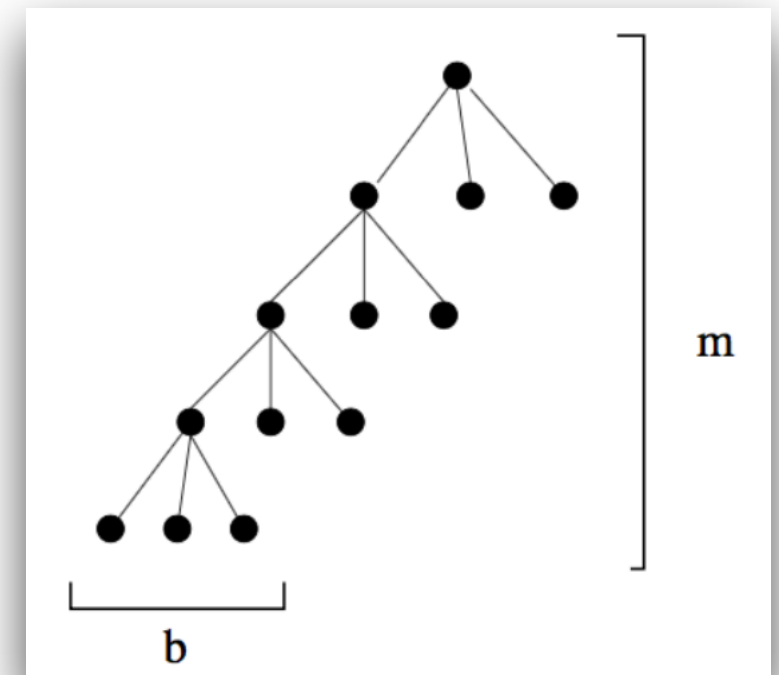
Completezza: Come già accennato, l'algoritmo non è completo nei casi in cui la soluzione si trova ad una profondità maggiore del limite stabilito.

Ottimalità: Questa strategia non può essere ottima nei casi in cui il limite stabilito sia minore della profondità massima (quindi sempre, altrimenti non sarebbe limitata).

Complessità temporale: La complessità è limitata dalla dimensione dello spazio degli stati. Sia l il limite prestabilito, nel caso pessimo la ricerca genererà $O(b^l)$ nodi.

Complessità spaziale: Rappresenta la peculiarità principale rispetto ai precedenti algoritmi. La ricerca deve memorizzare un solo cammino dalla radice ad un nodo foglia, insieme ai rimanenti nodi fratelli non espansi per ciascun nodo sul cammino. Una volta che un nodo è stato espanso, può essere rimosso dalla memoria non appena tutti i suoi discendenti sono stati esplorati completamente.

Per uno spazio degli stati con fattore di ramificazione b e profondità l , la complessità sarà di $O(bl)$.



Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca ad approfondimento iterativo

Questo tipo di ricerca ha l'obiettivo di identificare un limite ideale per la ricerca a profondità limitata, tuttavia non compromettendo costo, completezza e ottimalità.

Ricerca ad approfondimento iterativo: Con questa strategia, il limite viene incrementato progressivamente, finché un nodo obiettivo non è identificato.

```
function RICERCA-APPROFONDIMENTO-ITERATIVO(problema)  
    returns una soluzione o un fallimento  
  
    for profondità = 0 to  $\infty$  do  
        risultato  $\leftarrow$  RICERCA-PROFONDITÀ-LIMITATA(problema, profondità)  
        if risultato  $\neq$  fallimento then return risultato
```

Oltre a migliorare la ricerca in profondità, la ricerca ad approfondimento iterativo è analoga a quella in ampiezza, poiché ad ogni iterazione esplora completamente un livello di nodi prima di prendere in considerazione il successivo.

In generale, questo tipo di ricerca è il metodo di ricerca non informata da preferire quando lo spazio di ricerca è grande e la profondità della soluzione non è nota.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca ad approfondimento iterativo, performance

Da un punto di vista di performance, la ricerca a profondità limitata:

Completezza: In maniera simile alla ricerca in ampiezza, l'algoritmo è completo. Se il nodo obiettivo più vicino alla radice si trova alla profondità d , la ricerca lo troverà dopo aver espanso tutti i nodi che lo precedono.

Ottimalità: Se il costo di cammino è una funzione monotona non decrescente della profondità del nodo, allora possiamo dire che la ricerca è ottimo. Più in generale, però, l'algoritmo non lo è - restituisce la soluzione più vicina, indipendentemente dal costo.

Complessità temporale: Nel caso pessimo, saranno generati $O(b^d)$ nodi.

Complessità spaziale: Come per la ricerca in profondità limitata, la ricerca deve memorizzare un solo cammino dalla radice ad un nodo foglia, insieme ai rimanenti nodi fratelli non espansi per ciascun nodo sul cammino. Una volta che un nodo è stato espanso, può essere rimosso dalla memoria non appena tutti i suoi discendenti sono stati esplorati completamente. Per cui, la complessità sarà di $O(bd)$.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca bidirezionale

L'idea alla base è molto semplice: effettuare due ricerche in parallelo: la prima guidata dai dati, la seconda dall'obiettivo.

Ricerca bidirezionale: Strategia di ricerca in cui viene eseguita una prima ricerca in avanti dallo stato iniziale e l'altra all'indietro dallo stato obiettivo.

Perché? Semplicemente, perché $O(b^{d/2}) + O(b^{d/2}) < O(b^d)$, sia in termini di complessità temporale che spaziale.

Questo tipo di ricerca è implementata sostituendo il test obiettivo con un controllo che le frontiere delle due ricerche si intersechino: in tal caso, è stata trovata una soluzione.

Non è però così semplice implementarla. Cosa si intende per cercare “all'indietro”?

Predecessore: Definiamo predecessori di uno stato x tutti gli stati che hanno x come successore. Definire i predecessori non è sempre così immediato.

Cosa si intende per “obiettivo” nella ricerca “all'indietro”?

Consideriamo l'esempio del problema delle N regine. L'obiettivo è che “nessuna regina ne attacchi un'altra”: data questa descrizione astratta, è molto difficile definire cosa rappresenti l'obiettivo della ricerca all'indietro.



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

Lezione 4 - Algoritmi di ricerca non informata

